UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DSIC

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Dpto. de Sistemas Informáticos y Computación

Tipos Cociente en un Lenguaje Dependientemente Tipado

Trabajo Fin de Máster

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

AUTOR: Vegas, Luis

Tutor: Escobar Román, Santiago

CURSO ACADÉMICO: 2025-2026

Índice general

1. Introducción **1**
- 1.1. Motivación . . . 1
- 1.2. Objetivos . . . 1
- 1.3. Estructura del la tesis . . . 1

2. Fundamentos teóricos **2**
- 2.1. El Lambda cálculo . . . 2
  - 2.1.1. Booleanos . . . 3
  - 2.1.2. β-reducción . . . 3
  - 2.1.3. Variables libres y ligadas . . . 4
  - 2.1.4. Sustitución . . . 4
  - 2.1.5. Confluencia . . . 5
- 2.2. Teoría de Tipos . . . 8
  - 2.2.1. Tipos a la Curry . . . 16
  - 2.2.2. Tipos a la Church . . . 16
- 2.3. Lambda cálculo simplemente tipado . . . 16
- 2.4. Tipos Dependientes . . . 16
  - 2.4.1. Tipos Sigma . . . 19
- 2.5. Tipos Cociente . . . 20
- 2.6. Teoría de tipos de Martin-Löf . . . 23
- 2.7. Reescritura de términos en Maude . . . 23
  - 2.7.1. Sorts y lógica ecuacional ordenada . . . 24
  - 2.7.2. Módulos funcionales . . . 24
  - 2.7.3. Módulos de sistema y reescritura no determinista . . . 25
  - 2.7.4. Formalización de lenguajes y sistemas de tipos . . . 26
  - 2.7.5. Reflexión y meta-programación . . . 28

3. Un Lenguaje Dependientemente Tipado **29**
- 3.1. Sintaxis . . . 29
- 3.2. Sustitucion . . . 30
- 3.3. Normalizacion . . . 31
- 3.4. Verificacion de tipos . . . 32

4. Extension con tipos cociente **33**
- 4.1. Formacion . . . 33
- 4.2. Introduccion . . . 33
- 4.3. Eliminacion . . . 33

4.4. Computacion 33

5. Conclusiones **34**

A. Código Maude **35**

Capítulo 1

Introducción

1.1. Motivación

1.2. Objetivos

1.3. Estructura del la tesis

Capítulo 2

Fundamentos teóricos

2.1. El Lambda cálculo

El lambda cálculo fue inventado por Alonzo Church en la decada de los 80 con el proposito de capturar la nocion abstracta de la aplicabilidad de una funcion, sin embargo, conviene precisar en qué sentido. Una función puede entenderse de dos maneras: de forma extensional, como el conjunto de pares entrada–salida que la determina, o de forma intensional, como la regla o el proceso de cómputo que, dado un argumento, prescribe cómo obtener el resultado (Geuvers, 2014). Es esta segunda lectura, la de la función como algoritmo, la que el lambda cálculo formaliza (Barendregt, 1984; Geuvers, 2014). En su origen, Church concibió el sistema como parte de una fundamentación de la lógica y las matemáticas; dicha formulación resultó ser inconsistente, tal y como evidenciaron Kleene y Rosser, por lo que Church aisló su núcleo puramente computacional, libre de constantes lógicas, dando lugar al lambda cálculo tal y como se conoce hoy (Barendregt, 1984), siendo ampliamente adoptado como un sistema formal de lógica matemática.

Este sistema ocupa, además, un lugar cenral el diseño de lenguajes de programación. De forma análoga, la manera en la que se programa en el mismo es muy similar a cómo se programa en un lenguaje funcional hoy en dia. De hecho, lenguajes como Haskell, Agda, la familia de lenguajes basados en ML, y LISP, estan ampliamente inspirados en un conjunto del lambda calculus y, pueden entenderse, en buena medida, como extensiones dotadas de azucar sintactico, sistemas de tipos y otras primitivas (Pierce, 2002).

El lambda calculo es un lenguaje muy potente y, a su vez, sumamente simple. Un término puede estar compuesto, o bien, formarse, de tres elementos: variables, aplicacion y abstracción lambda. De forma inductiva, fijado un conjunto infinito numerable de variables V, el conjunto Λ de los términos lambda se define como el menor conjunto cerrado bajo las siguientes reglas (Barendregt, 1984; Pierce, 2002):

$$
\begin{aligned}
M := \quad & x && (1)\\
\mid \quad & \lambda x.\, M && (2)\\
\mid \quad & M\ N && (3)
\end{aligned}
$$

Donde, (1) es un conjunto infinito de variables, tal que si $x, y, z \in V$ y $V \in \Lambda$, entonces $V \in \Lambda$. (2) es una abstracción lambda, si $x \in V$ y además $M \in \Lambda$, entonces $\lambda x.\, M \in \Lambda$. (3) Es una aplicación de dos términos, si $M, N \in \Lambda$ entonces $MN \in \Lambda$.

El lambda calculo es lo suficientemente potente para permitirnos codificar simples tipos de datos o, al menos, emular el comportamiento que obtendriamos de los mismos en un sistema de tipos mas avanzado.

2.1.1. Booleanos

El tipo booleano es un tipo habitado por solo dos posibles terminos: $True$ o $False$. Pese a que el lambda calculo sin tipos no posee una nocion de verdad o falsedad, es posible, aun asi, codificar los booleanos, como se muestra (citar bargendregt):

Ejemplo 2.1. $\text{True} \equiv \lambda xy.\, x, \quad \text{False} \equiv \lambda xy.\, y$

A continuación se detallan propiedades de sumo interés que articulan, posteriormente, los fundamentos del lenguaje que describe esta tésis: la distinción entre variables libres y ligadas, junto con la α-equivalencia que de ella se deriva; la operación de sustitución que evita la captura de variables; y la β-reducción como noción primaria de cómputo. Se mencionará igualmente la η-conversión, que captura la extensionalidad de las funciones.

2.1.2. β-reducción

La noción de evaluacion de un término viene dada por beta-reducción, donde la notación $\to_\beta$ es empleada. La $\to_\beta$ está basada en la *sustitución*, formalmente expresada como $M[x := N]$, significando que M es un término donde en su cuerpo, x, ha sido reemplazado por N.

Por tanto, una expresión de la forma $(\lambda x.\, M)\, N$ puede ser reescrita como $M[\, x := N]$, donde toda ocurrencia de x en M (estando x libre en M), debe ser reemplazada por N (Geuvers, 2014).

Ejemplo 2.2. $(\lambda x.\, x + 1)\, 5$ reduce a $(x + 1)[x := 5] = 5 + 1 = 6$.

Ejemplo 2.3. La función identidad aplicada a un término cualquiera devuelve dicho término: $(\lambda x.\, x)\, y \to_\beta x[x := y] = y$.

Ejemplo 2.4. La β-reducción puede encadenarse, reduciendo paso a paso hasta que no queden más redexes. considérese el combinador $K \equiv \lambda x.\, \lambda y.\, x$ aplicado a dos argumentos:

$$\begin{aligned}(\lambda x.\, \lambda y.\, x)\, a\, b &\to_\beta (\lambda y.\, x)[x := a]\, b \\ &= (\lambda y.\, a)\, b \\ &\to_\beta a[y := b] = a.\end{aligned}$$

Nótese que el segundo argumento, b, es descartado, pues y no ocurre en el del lambda.

2.1.3. Variables libres y ligadas

La abstracción $\lambda x.\, M$ introduce un *binder* sobre la variable x, cuyo *alcance* es el cuerpo de M. Decimos, pues, que una ocurrencia de una variable x está *ligada* cuando aparece dentro del alcance del λx que la introduce; en caso contrario, dicha ocurrencia se dice *libre*.

Conviene resaltar que una misma variable puede ocurrir libre y ligada dentro de un mismo término. En $(\lambda x.\, x)\, x$, por ejemplo, la primera ocurrencia de x está ligada por el λx, mientras que la segunda, externa a la abstracción, permanece libre. Formalmente, el conjunto de variables libres de un término, denotado $FV(M)$, se define de forma inductiva sobre la estructura del término (Pierce, 2002):

$$
\begin{aligned}
&(1)\ FV(x) = \{x\},\\
&(2)\ FV(\lambda x.\, M) = FV(M) \setminus \{x\},\\
&(3)\ FV(M\, N) = FV(M) \cup FV(N).
\end{aligned}
$$

(1) indica que una variable, por sí sola, es libre. (2) establece que al abstraer sobre x, esta deja de ser libre en $\lambda x.\, M$, pues queda ligada por el nuevo binder; por ello se retira del conjunto. (3) recoge que las variables libres de una aplicación son, simplemente, la unión de las libres de cada subtérmino M y N respectivamente. Un término M tal que $FV(M) = \emptyset$ se denomina *cerrado*.

Por último, el nombre concreto de una variable ligada carece de relevancia: lo único que importa es la estructura de ligaduras que establece. Así, $\lambda x.\, x$ y $\lambda y.\, y$ denotan exactamente la misma función identidad. A esta noción, que identifica términos que difieren únicamente en el nombre de sus variables ligadas, se le conoce como α-*equivalencia*, y se denota $=_\alpha$. El proceso de renombrar una variable ligada para obtener un término α-equivalente recibe el nombre de α-*conversión*, y será fundamental para definir correctamente la sustitución bajo un binder.

2.1.4. Sustitución

Anteriormente, definimos la sustitución en términos de $\to_\beta$ reducción, sin embargo, la sustitucion posee en sí reglas de computo que valen la pena considerar pues, dependiendo del término que se esté evaluando, la sustitución tendrá, o no, efecto alguno.

Se definen, pues, las reglas de computo de la sustitucion de acuerdo a lo discutido por Pierce (2002). Más formalmente:

$$
\begin{aligned}
&(1)\ x[x := N] \equiv N,\\
&(2)\ y[x := N] \equiv y \text{ if } x \not\equiv y,\\
&(3)\ (PQ)[x := N] \equiv (P[x := N])(Q[x := N]),\\
&(4)\ (\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y \to z}[x := N]),\ \text{si } \lambda z.\, P^{y \to z} \text{ es un } \alpha\text{-variante de}\\
&\quad \lambda y.\, P \text{ tal que } z \notin FV(N).
\end{aligned}
$$

Para (1), sustituyendo x, por N dentro de x, siendo x el término que buscamos sustituir, obtenemos N. De forma análoga, sustituyendo cada ocurrencia de x en el término y, por N, si x e y no son iguales, el término intacto, y, será retornado. El caso de (3), tiene sentido cuando P es una abstracción aplicada a un término Q. Concretamente, se reduce a la sustitución de x por N, en ambos P y Q. (4) contempla la sustitución bajo un binder. Lo que se busca es sustituir x por N dentro del cuerpo P de la abstracción $\lambda y.\, P$. Sin embargo, si procediéramos de forma ingenua y N contuviera a y como variable libre, dicha y quedaría equivocadamente atrapada por el binder λy, alterando por completo el significado del término. A este fenómeno se le conoce como *captura de variables*.

Para evitarlo, antes de sustituir se renombra la variable ligada y por una variable fresca z, mediante α-conversión, escogida de modo que $z \notin FV(N)$ (y, por extensión, que no genere conflicto con las variables ya presentes en P). Solo entonces se procede a sustituir x por N dentro del cuerpo ya renombrado. De esta manera, la sustitución preserva el significado del término original, razón por la cual se la describe como una sustitución *que evita la captura* (Pierce, 2002).

Ejemplo 2.5. Supóngase la sustitución $(\lambda y.\, x)[x := y]$. Si sustituyéramos directamente, obtendríamos $\lambda y.\, y$, es decir, la identidad: la y libre que introdujimos habría sido capturada por el binder. Renombrando primero la variable ligada a una fresca z, obtenemos $(\lambda z.\, x)[x := y] = \lambda z.\, y$, que es el resultado correcto, pues la y permanece libre.

2.1.5. Confluencia

Anteriormente señalamos cómo la β-reducción emula, en cierto sentido, el proceso de cómputo de una funcion dada: evaluado un término, aspiramos a obtener su forma normal, esto es, un término en el que ya no resta redex alguno que nos permita seguir reduciendo (Barendregt, 1984; Pierce, 2002).

Ahora bien, el orden en que se contraen tales redexes no es indiferente, pues puede influir en si el cómputo termina o, mejor dicho, en si alcanza una forma normal. Surge así una pregunta: dado un término t del que divergen dos reducciones, $t \to_\beta^* t_1$ y $t \to_\beta^* t_2$, en uno o varios pasos, ¿es posible reconciliar ambas ramas en un término común u alcanzable tanto desde t_1 como desde t_2? Cuando tal u existe para cualquier par de reducciones divergentes, decimos que la reducción es *confluente* sobre t.

En el presente apartado describimos con detenimiento la propiedad de confluencia y, exponémos también, por qué la β-reducción la satisface.

La confluencia, conocida también como teorema de Church-Rosser, es una propiedad de los sistemas de reescritura que caracteriza cuándo dos términos, derivados de un término inicial, por reducción en uno o más pasos, convergen a un mismo resultado. Su relevancia radica en que el resultado del cómputo queda determinado con independencia del orden en que se ejecuten los pasos de reducción (Barendregt, 1984).

Teorema 2.1 (Church-Rosser)**.** *Sea M un término del λ-cálculo. Si $M \to_\beta^* N_1$ y $M \to_\beta^* N_2$, entonces existe un término N_3 tal que $N_1 \to_\beta^* N_3$ y $N_2 \to_\beta^* N_3$ (Barendregt, 1984, p. 62).*

Conviene precisar que el teorema habla de la *clausura reflexivo-transitiva* de la reducción, $\to_\beta^*$, es decir, de cero o más pasos de β-reducción. Es importante resaltar que la β-reducción no cumple la versión más fuerte de la propiedad, conocida como *propiedad del diamante* en un solo paso: si de M contraemos dos redexes distintos en un único paso para obtener N_1 y N_2, no siempre existe un N_3 alcanzable desde ambos en un solo paso. La razón es la duplicación, pues un redex puede copiar a otro cuando el argumento contiene, a su vez, redexes, de modo que un lado de la bifurcación necesita varios pasos para reabsorber las copias que el otro lado ya había contraído. Lo que el teorema de Church-Rosser garantiza es la versión global, sobre $\to_\beta^*$, que sí se sostiene.

Ademas, por simplicidad, hasta ahora hemos de forma ligera abusado del término de confluencia. Resulta importante separar dos nociones de la misma: la confluencia local y la confluencia global. Decimos que un sistema es *localmente confluente* cuando, partiendo de M y dando un paso hacia N_1 y otro hacia N_2, existe un término común alcanzable desde ambos en cero o máas pasos. La confluencia local es más débil que la confluencia global, y la implicación de la primera a la segunda no siempre se cumple, pues solo se obtiene bajo hipótesis adicionales, como la terminación, por tanto, una relacion (sistema?) terminante es confluente, si es localmente confluente. de acuerdo al lema de Newman. (Baader, 2012; Barendregt, 1984).

Lema 2.2. *1. M es una forma normal si M no contiene más redexes.*

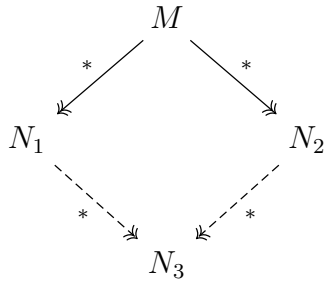
2. M tiene una forma normal si existe un N, en forma normal, tal que $M =_\beta N$. Tal N es la forma normal de M.

Una consecuencia inmediata, y quizá la más útil en la práctica, es la unicidad de la forma normal. Si un término admite forma normal, esta es única salvo α-equivalencia.

Corolario 2.3. *Si M posee una forma normal, esta es única salvo α-equivalencia. En efecto, supóngase que N_1 y N_2 son ambas formas normales de M, de modo que $M \to_\beta^* N_1$ y $M \to_\beta^* N_2$. Por el teorema de Church-Rosser existe N_3 con $N_1 \to_\beta^* N_3$ y $N_2 \to_\beta^* N_3$. Pero N_1 y N_2 están en forma normal, luego no contienen redex alguno y no pueden reducirse en pasos no triviales, de donde $N_1 =_\alpha N_3 =_\alpha N_2$.*

Este corolario, por un lado, justifica que hablemos de *la* forma normal de un término, en singular, cuando esta existe. Por otro, garantiza que el valor de un cómputo, entendido como su forma normal, queda determinado por el término de partida y no por la estrategia de evaluación escogida, de modo que el orden en que decidamos contraer los redexes solo puede afectar a si terminamos, mas no al resultado al que llegamos.

La confluencia se representa mediante el siguiente diagrama, cuya forma de rombo explica que la propiedad reciba también el nombre de *propiedad del diamante*. El vértice superior es el término de partida M, las dos diagonales descendentes son las reducciones divergentes hacia los terminos N_1 y N_2, y las diagonales punteadas representan los pasos que aún restan, garantizados por el teorema, para reconciliar ambas ramas en un término común N_3.



Ejemplo 2.6. Considérese el término $M \equiv (\lambda x.\, x\, x)\, ((\lambda y.\, y)\, z)$, que presenta dos redexes: el externo, que aplica $\lambda x.\, x\, x$ a su argumento, y el interno, $(\lambda y.\, y)\, z$. Contrayendo primero el redex externo, el argumento se duplica y obtenemos $N_1 \equiv ((\lambda y.\, y)\, z)\, ((\lambda y.\, y)\, z)$, que aún requiere dos pasos para resolver las dos copias del redex interno. Contrayendo primero el redex interno, en cambio, llegamos en un solo paso a $N_2 \equiv (\lambda x.\, x\, x)\, z$. Ambas ramas convergen en $z\, z$:

Obsérvese cómo la rama izquierda necesita más de un paso para alcanzar la confluencia, lo cual ilustra por qué el teorema se enuncia sobre $\to_\beta^*$ y no sobre la reducción en un único paso.

De aquí emerge un problema de naturaleza distinta: la terminación. La terminación determina si el cómputo de un término concluye o no. En el λ-cálculo, una secuencia de reducciones puede no detenerse, entrando en un ciclo infinito de pasos que impide alcanzar forma normal alguna, como ilustra el siguiente ejemplo.

Ejemplo 2.7. El ejemplo canónico de término sin forma normal es el combinador $\Omega \equiv (\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$. Al contraer su único redex, el argumento $\lambda x.\, x\, x$ se sustituye en el cuerpo $x\, x$, devolviendo de nuevo Ω:

$$(\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \to_\beta (x\, x)[x := \lambda x.\, x\, x] = (\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \equiv \Omega.$$

La reducción se cierra pues, sobre sí misma:

$$\Omega \equiv (\lambda x.\, x\, x)\, \overset{\beta}{\underset{\leftarrow}{(\lambda x.\, x\, x)}}$$

El término es trivialmente confluente, ya que su única reducción conduce siempre a sí mismo, y sin embargo carece de forma normal. Este ejemplo evidencia que la confluencia y la terminación son propiedades independientes: un sistema puede ser confluente sin ser terminante.

El ejemplo de Ω pone de manifiesto que el λ-cálculo sin tipos no es terminante, pues existen términos que carecen de forma normal y cuya reducción no se detiene

jamás. La terminación, entendida como normalización fuerte, se recupera al imponer una disciplina de tipos sobre los términos. En particular, el λ-cálculo simplemente tipado sí es fuertemente normalizante, propiedad que abordaremos en un capitulo posterior, al tratar dicho sistema.(Geuvers, 2014; Pierce, 2002).

2.2. Teoría de Tipos

La teoria de tipos es una rama de las matematicas y la logica que tiene como objeto formalizar no solo terminos matematicos, tales como una variable x o una funcion cualquiera f y las operaciones entre ambas, si no tambien, la idea de que cada termino forma parte o tiene un tipo concreto, es decir, que el tipo de una variable $x : \mathbb{N}$ en el dominio de los numeros naturales, es completamente distinguible al tipo de una funcion $f : \mathbb{N} \to \mathbb{N}$.

Los tipos, pese a que son similares a los conjuntos, son ligeramente distintos. Por conjunto, entiéndase ocmo una colección de elementos, similares o no, que poseen una propiedad concreta:

$$C = \{rojo, amarillo, azul\} \tag{2.1}$$

de donde es posible extenderlos y razonar sobre más relaciones de pertenencia y formar, por ejemplo, conjuntos dentro de otros conjuntos y definir que un elemento pertenece a varios conjuntos a la vez:

$$C' = \{\text{verde}, \text{naranja}\} \cup C \tag{2.2}$$

Cosa que, en los tipos, no es posible pues, un termino es de (o pertenece a) un tipo unico. Otra caracteristica que los distingue, es que los conjuntos dan informacion semantica y, los tipos por otro lado, proveen informacion sintactica de los objetos. Por ejemplo, comunmente en la teoria de conjuntos definimos una relacion de pertenencia como sigue:

$$4 \in \{\, n \in \mathbb{N} | x > 5 \,\} \tag{2.3}$$

En la teoria de tipos, un juicio de tipado, sigue la siguiente notacion que, a lo largo de la presente tesis estarémos empleando:

$$3 + 1 : \mathbb{N} \tag{2.4}$$

Esto puede interpretarse, desde el lente de teoria de tipos, como $3 + 1$ es un término en los naturales, o bien, de tipo $\mathbb{N}$. Aún así, las mismas operaciones aplicadas en la teoria de conjuntos son trasladables a la teoria de tipos. Tómese como ejemplo la unión disjunta de conjuntos, definida formalmente como la unión disjunta $A \sqcup B$ de dos conjuntos, es un conjunto A indexado por B:

$$\bigcup_{i \in I} \{(x, i) : x \in A_i\} \tag{2.5}$$

Esta operacion resulta en el tipo de dato algebraico $\tau_1 + \tau_2$, donde el valor concreto del tipo puede bien ser la proyeccion del lado izquiero o el derecho.

La noción de tipos se atribuye a Bertrand Russell, quien desarrolló la teoría de tipos en respuesta a las contradicciones que, a comienzos del siglo XX, aquejaban a los fundamentos de la matemática. El contexto es el del programa logicista y la teoría de conjuntos naciente. Hacia finales del siglo XIX, Georg Cantor había desarrollado la teoría de conjuntos como un marco general en el que toda la matemática podía, en principio, expresarse, y Gottlob Frege perseguía un objetivo todavía más ambicioso en sus *Grundgesetze der Arithmetik*: reducir la aritmética a la lógica pura. Ambos proyectos descansaban, de forma más o menos explícita, sobre el principio de comprensión irrestricta, según el cual toda propiedad bien definida determina un conjunto, el de todos los objetos que la satisfacen (Coquand, 2022).

Russell advirtió en 1901 que ese principio conducía a una contradicción. Si toda propiedad define un conjunto, entonces la propiedad de no pertenecer a sí mismo define el conjunto de todos los conjuntos que no son miembros de sí mismos:

$$R = \{\, x \mid x \notin x \,\} \tag{2.6}$$

Preguntarse si R pertenece o no a sí mismo lleva, por tanto, a una contradicción lógica, pues por la propia definición de R se tiene que:

$$R \in R \iff R \notin R \tag{2.7}$$

es decir, R es miembro de sí mismo exactamente cuando no lo es. La gravedad del hallazgo se aprecia mejor al recordar que Russell se lo comunicó a Frege por carta en 1902, cuando el segundo volumen de los *Grundgesetze* estaba ya en imprenta; Frege reconoció, en un apéndice, que la paradoja socavaba el fundamento mismo de su sistema.

Lo que Russell sugeria pues, es que estas paradojas comparten una misma estructura, todas involucran una definición que se refiere a una totalidad de la que el objeto definido forma parte. A esto lo llamó el principio del círculo vicioso, que prohíbe que una entidad se defina en términos de una colección a la que ella misma pertenece. La expresión $x \in x$, sobre la que se construye R, es precisamente una de estas definiciones circulares, o autoreferencia.

Para evitarlas, Russell propuso organizar los objetos en una jerarquía estratificada. En el nivel más bajo están los individuos; en el siguiente, los predicados (o clases) de individuos; después los predicados de predicados, y así sucesivamente. Esta distinción es la que la teoría de tipos internaliza: a cada objeto se le asigna un tipo, y una función proposicional solo puede aplicarse a argumentos del tipo inmediatamente inferior. Bajo esta restricción, la fórmula $x \in x$ ni siquiera llega a estar bien y es rechazada, ya que la pertenencia exige que su argumento sea de un tipo menor que el del conjunto. El conjunto R no puede construirse, y la paradoja desaparece por construcción. De este modo, conjuntos grandes y pequeños residen en niveles distintos de la teoría.

La formulación original de Russell, desarrollada junto a Whitehead en *Principia Mathematica*, era la teoría ramificada de tipos, que sumaba a la estratificación por tipos una segunda por órdenes para neutralizar también las paradojas semánticas, al precio de requerir el discutido axioma de reducibilidad. Ramsey observó más tarde que las paradojas estrictamente lógicas o conjuntistas quedan resueltas ya con la jerarquía simple de tipos, sin necesidad de los órdenes ni del axioma de reducibilidad. Es esta teoría simple de tipos la que, años después, Church retomaría como base de su lambda calculo tipado, y la que enlaza directamente con el objeto de este trabajo.

En términos más generales, los tipos son ubicuos en computación, y numerosos lenguajes tienen su propio sistema de tipos, unos mas avanzados que otros y, otros más simples. Tan simples que, la manipulación de tipos es casi perceptible o inexitstente. Lenguajes con esta capacidad de omitir tipos son conocidos como lenguajes dinámicamente tipados, donde en casos, es completamente opcional tipar expresiones y en caso contrario, el compilador hace lo mejor que puede para inferir un tipo adecuado para una expresión. Haskell por ejemplo, es un lenguaje de alto nivel, ampliamente utilizado por grupos de investigación y académicos, su sistema de tipos esta inicialmente basado en el lambda calculus simplemente tipado, también llamado *System F*, que abordaremos en capítulos posteriores a este. Sin embargo, hoy en día su sistema de tipos ha madurado lo suficiente como para dejar de considerarle un lenguaje con un sistema de tipos simple. C++ es un claro ejemplo de un sistema de tipos, quizá, mucho más complejo, pero sumamente rico, que facilita posteriormente, una distincion semántica y rechaza programas que son imposibles de evaluar correctamente. OCaml en principio, fue de los lenguajes que llego a explorar más a fondo la teoria de tipos y la expresividad de los mismos como funcionalidades del lenguaje, siendo de los primeros en implementar tipos polimórficos, es decir, tipos que pueden tomar la forma de otros tipos. Finalmente, un sistema de tipos permite al lenguaje alcanzar capacidades de expresividad bastante amplias y, a su vez, evitar que programas erronoes se ejecuten.

Dentro de los problemas que intenta resolver la teoria de tipos, no se encuentra solo encontrar un tipo para un término concreto, sino también, determinar (si lo hay) un término para un tipo y si un término esta bien tipado (o es un tipo legal) dentro de un contexto (Geuvers, 2014). Listamos pues, algunos de los problemas que se abordan en la teoria de tipos:

1) Tipabilidad: este problema formula la pregunta: ¿existe un tipo bien tipado para el término en concreto en el contexto?

$$\Gamma \vdash a :?$$

No siempre existe un tipo bien definido para un término. Las derivaciones o juicios de tipado sirven justamente para determinar logicamente cual tipo es el adecuado o, correctamente, inferir el tipo del término. No encontrar dicho tipo, por supuesto, ocasionaría un error de tipado.

2) Type-checking: determina si el tipo de un término en un contexto Γ cualquiera, es correcto o bien tipado.

$$\Gamma \overset{?}{\vdash} a : A$$

Sea $\Gamma = \{b : B, U : *\}$, si A no es un tipo bien formado en Γ, el sistema de tipos lo rechaza. Esto es claro, pues, no hay una regla que permita derivar que $a : A$ esta bien formado en Γ.

3) Habitabilidad: el problema de la habilitabilidad (también llamado term finding), formula la pregunta: ¿existe un término, dado un tipo bien definido y un contexto concreto?

$$\Gamma \overset{?}{\vdash} ? : A$$

Nótese que los problemas anteriormente mencionados son decibibles en $\lambda \rightarrow$, es decir, para cada una de estas cuestiones existe un algoritmo posible que produce un resultado en el lambda calculo simplemente tipado. En sistemas de tipos más complicados, sin embargo, la búsqueda de un término llega a ser un problema, pues tales sistemas no logran encontrar un habitante del tipo en el contexto, siendo problemas no decidibles (Barendregt et al., 2013).

Se empleará, a lo largo de la tesis, la siguiente notación para definir las derivaciones en la teoría de tipos, donde la parte superior es una lista de premisas y la parte inferior, la conclusión o los consecuentes:

$$\frac{premisa_1 ... \quad premisa_n}{consecuente_1 ... consecuente_n} \tag{2.8}$$

Para $n \geq 0$. Las reglas pueden leerse bien de arriba hacia abajo (como *teniendo que, podemos concluir que*) o de abajo hacia arriba (como *dado que se cumple el consecuente, las premisas deben cumplirse*). Adicionalmente, las variables presentes en premisas y consecuentes, estan universialmente quantificadas a menos de que se indique lo contrario. Por tanto,

$$\frac{}{\forall x, P(x)} \tag{2.9}$$

Donde $\forall$ es implícito.

$$\frac{\Gamma \vdash A \quad \Gamma, a : A}{\Gamma \vdash a : A} \tag{2.10}$$

Que puede leerse como: *si, dado que en el contexto* Γ*,* A *es un tipo bien formado y, ademas, en el mismo contexto* Γ *extendido con* $a : A$*, entonces podemos inferir que* a *es de tipo* A *en* Γ.

Dentro de la teoría de tipos existen seis tipos de juicios que seran de utilidad en la medida en que estudiemos los sistemas formales en este trabajo:

$\vdash \Gamma \; ctx$	Γ es un contexto valido
$\Gamma \vdash \sigma \; Type$	σ es un tipo
$\Gamma \vdash M : \sigma$	M es un término de tipo σ en Γ
$\vdash \Gamma \equiv \Delta$	$\Gamma \; y \; \Delta$ son contextos definicionalmente iguales
$\Gamma \vdash \sigma \equiv \tau \; Type$	$\sigma \; y \; \tau$ son tipos definicionalmente iguales en Γ
$\Gamma \vdash M \equiv N : \sigma$	$M \; y \; N$ son terminos definicionalmente iguales, de tipo σ, en Γ

Adicionalmente, independientemente del sistema de tipos que se describa, las mismas reglas estructurales se adoptan, como describe Hofmann, 2009.

$$\frac{}{\vdash \emptyset} \text{ C-Emp} \qquad \frac{\Gamma \vdash \sigma}{\vdash, \Gamma \; x : \sigma} \text{ C-Ext} \tag{2.11}$$

$$\frac{\vdash \Gamma = \Delta \quad \Gamma \vdash \sigma = \tau \; \mathsf{Type}}{\Gamma, x : \sigma \equiv \Delta, \; y : \tau} \text{ C-Eq} \tag{2.12}$$

$$\frac{\vdash \Gamma, x : \sigma}{\Gamma \vdash x : \sigma} \text{ Var} \tag{2.13}$$

Asimismo, para las reglas que expresan la igualdad definicional como una relación

de equivalencia, tenemos que:

$$\frac{\vdash \Gamma \ ctxt}{\vdash \Gamma = \Gamma \ ctxt} \text{ C-Eq-R} \tag{2.14}$$

$$\frac{\vdash \Gamma = \Delta \ ctxt}{\vdash \Delta = \Gamma \ ctxt} \text{ C-Eq-S} \tag{2.15}$$

$$\frac{\vdash \Gamma = \Delta \ ctxt \quad \vdash \Delta = \Theta \ ctxt}{\vdash \Gamma = \Theta \ ctxt} \text{ C-Eq-T} \tag{2.16}$$

$$\frac{\Gamma \vdash \sigma \ \mathsf{type}}{\Gamma \vdash \sigma = \sigma \ \mathsf{type}} \text{ Ty-Eq-R} \tag{2.17}$$

$$\frac{\Gamma \vdash \sigma = \tau \ \mathsf{type}}{\Gamma \vdash \tau = \sigma \ \mathsf{type}} \text{ Ty-Eq-S} \tag{2.18}$$

$$\frac{\Gamma \vdash \sigma = \tau \ \mathsf{type} \quad \Gamma \vdash \tau = \rho \ \mathsf{type}}{\Gamma \vdash \sigma = \rho \ \mathsf{type}} \text{ Ty-Eq-T} \tag{2.19}$$

$$\frac{\Gamma \vdash M : \sigma}{\Gamma \vdash M = M : \sigma} \text{ Tm-Eq-R} \tag{2.20}$$

$$\frac{\Gamma \vdash M = N : \sigma}{\Gamma \vdash N = M : \sigma} \text{ Tm-Eq-S} \tag{2.21}$$

$$\frac{\Gamma \vdash M = N : \sigma \quad \Gamma \vdash N = O : \sigma}{\Gamma \vdash M = O : \sigma} \text{ Tm-Eq-T} \tag{2.22}$$

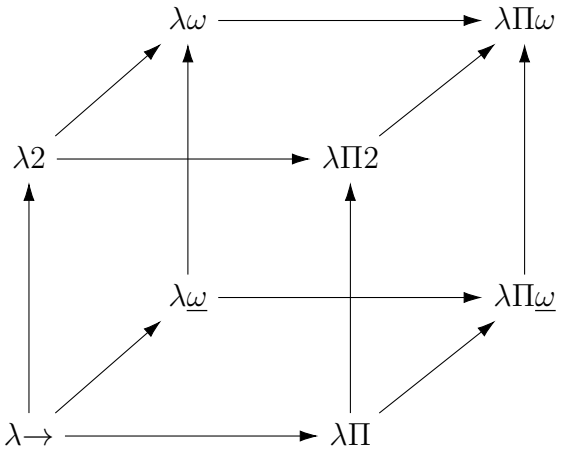
$$\frac{\Gamma \vdash M : \sigma \quad \vdash \Gamma = \Delta \ ctxt \quad \Gamma \vdash \sigma = \tau \ \mathsf{type}}{\Delta \vdash M : \tau} \text{ Tm-Conv} \tag{2.23}$$

$$\frac{\vdash \Gamma = \Delta \ ctxt \quad \Gamma \vdash \sigma \ \mathsf{type}}{\Delta \vdash \sigma \ \mathsf{type}} \text{ Ty-Conv} \tag{2.24}$$

A lo largo de los años numerosos esfuerzos se han realizado para desarrollar distintos sabores (sabores es la palabra? quiza extensiones) de la teoría de tipos y poder dar suficiente poder expresivo al lambda calculo. Estos sistemas de tipos fueron explorados por H. Barendregt en el Cubo Lambda (Barendregt, 1991b), para investigar las diferentes dimensiones en las que puede extenderse el sistema formal. En su paper describe como cada dimension del cubo representa una dependencia entre terminos y tipos, y sus posibles combinaciones.

Entiendase por dependencia como la dependencia mutua, valga la reduncancia, que existe entre tipos y terminos: terminos que dependen de otros terminos, terminos dependientes de tipos, tipos dependientes de terminos, y tipos dependientes de otros tipos. Los ocho vertices del cubo, consecuencia de combinar cada posible dimension, resultan en diferentes sistemas de tipos y niveles de expresividad. Concretamente,

abordaremos en el presente trabajo, la extension del vertice inferior derecho, que corresponde a la extension de tipos dependientes, concreamente $\lambda\Pi$, o refiérase también en la literatura como λP.



la dependencia que define cada vertice, es descrita por la siguiente tabla:

Sistema	Conjunto de reglas			
$\lambda\!\rightarrow$	$(*,*)$			
$\lambda 2$	$(*,*)$	$(\Box,*)$		
λP	$(*,*)$		$(*,\Box)$	
λP2	$(*,*)$	$(\Box,*)$	$(*,\Box)$	
$\lambda\underline{\omega}$	$(*,*)$			$(\Box,\Box)$
$\lambda\omega$	$(*,*)$	$(\Box,*)$		$(\Box,\Box)$
λP$\underline{\omega}$	$(*,*)$		$(*,\Box)$	$(\Box,\Box)$
λP$\omega = \lambda$C	$(*,*)$	$(\Box,*)$	$(*,\Box)$	$(\Box,\Box)$

Es necesaria una explicacion a cada conjunto de reglas y nivel de expresividad que permite cada sistema. $\lambda\rightarrow$ es posiblemente, el más común. Formalmente este sistema es conocido como el lambda cálculo simplemente tipado que se abordará con más dedicación en un capítulo aparte, donde para construir una abstracción lambda, basta con proveer otro término. Por tanto FM es, por ejemplo, un término en Λ donde F depende de M.

Ejemplo 2.8.

$$(\lambda x : \mathbb{N}.t)3 \quad \longrightarrow_\beta \quad 3 \tag{2.25}$$

$\lambda 2$ es conocido en la literatura como *System F*. Es el lambda cálculo de segundo orden, o polimórfico. Fue descubierto por Girard y Reynolds en 1972 en el cual se introduce el cuantificador universal en el cálculo y poder cuantificar sobre tipos. A parte de ello, algunas implementaciones extienden el sistema introduciendo kinds, el tipo de todos los tipos. Un gran número de lenguajes funcionales, como Haskell, están inspirados en este sistema. Además de esto, en $\lambda 2$ se presenta la noción de variables

de tipos, con las cuales es posible cuantificar sobre un rango de tipos, permitiendo así, la suficiente expresividad al lambda calculo para definir, por instancia, la función identidad polimórfica.

Ejemplo 2.9.

$$(\Lambda\alpha\lambda x : \alpha.x)\, 3 \longrightarrow_\beta 3$$
$$(\Lambda\alpha\lambda x : \alpha.x)\, \mathit{true} \longrightarrow_\beta \mathit{true}$$

Donde Λ representa el cuantificador universal sobre los tipos del lenguaje.

$\lambda\underline{\omega}$ nos permite definir tipos que dependen de otros tipos. Un ejemplo concreto en un lenguaje de programación, es la definición inductiva de un tipo que depende de un otro para poder usarse a lo largo del programa. Los arboles binarios o los números de Peano son un ejemplo claro de esta extensión.

Ejemplo 2.10.

data Tree a = Empty | Node a (Tree a) (Tree a)

λP representa los tipos que dependen de términos. Los tipos dependientes, como se les referirá a lo largo de este etrabajo, extienden el sistema de tipos con familias de tipos, o tipos indexados por los elementos de otros tipos. La siguiente regla de inferencia formaliza esta idea donde x puede o no, aparecer en B. Observese que de no aparecer x en B, estariamos involucrandonos con el tipo de función $A \to B$.

$$\frac{\Gamma \vdash A : *,\ x : A \quad \Gamma \vdash B : A}{\Gamma \vdash \Pi x : A.Bx} \tag{2.26}$$

Tómese como ejemplo *Vec A n* donde la longitud n del vector, depende de cierto parámetro. Decimos, por tanto, que *Vec A n* es un tipo dependiente o bien una familia de tipos indexado por n. Asimismo, la linea con la que se distinguen términos y tipos se hace más difusa y, en consecuencia, ambos comparten el mismo espacio sintactico. Por ello, encontramos la constante $*$ que representa el *sort* en el que habitan todos los tipos, por tanto, la siguiente derivación es posible en λP: $A : * \vdash (A \to A) : *$, que expresa que *dado que A es un tipo bien formado, (A → A), es entonces, un tipo* (Barendregt, 1991b). El símbolo $\Box$ no es propiamente un término en la sintaxis, en cambio, es un símbolo en la meta-teoría para afirmar que se tiene una derivación de kinds válida, por tanto, se puede entender $\Gamma \vdash (\kappa : \Box)$ mediante la siguiente lectura: *k es un kind bien formado dado el contexto* Γ.

2.2.1. Tipos a la Curry

2.2.2. Tipos a la Church

2.3. Lambda cálculo simplemente tipado

2.4. Tipos Dependientes

Los tipos dependientes son, en principio, tipos que dependen de términos. Antes de adentrarnos en la teoría de tipos dependientes, es necesario dedicar de forma breve una introduccion a los sistemas de tipos puros, framework de donde parte esta formalización lógica. Un PTS es un framework para especificar y formalizar el lambda cálculo y extender su teoría. Fue propuesto inicialmente por Berardi y Terlow como una generalizacion del cubo lambda de Barendregt (1991a). A diferencia del lambda calculo tradicional, un PTS dispone de pseudo-términos que heredan, en gran parte, sintáxis del λ calculo simplemente tipado que hasta ahora conocemos.

El conjunto de pseudo-términos, se define como sigue:

Definición 2.1. Pseudo-términos de un PTS

$$T = T \mid T\,T \mid S \mid \lambda V : T.\,T \mid \Pi V : T.\,T$$

Asimismo, resalta Barthe and Coquand algunas caracteristicas a tomar en cuenta, que ofrecen una imagen mas amplia de la definición de un sistema de tipos puro.

1. Se utiliza la misma convencion para extraer el conjunto de variables libres en un pseudo-termino $FV(M)$. Adicional a ello, se escribirá $A \to B$ para todo tipo de función en el que x no aparezca libre en B. Y se usará $\Pi x : A.B(x)$, en caso contrario.

2. Los sistemas de tipos puros, describen un λ calculo a la Church, es decir, las abstracciones lambda llevan consigo el dominio de las variables ligadas y, como consecuencia de ello, la computabilidad en un sistema de tipos puro es decidible.

3. Los sistemas de tipos puros proveen los cimientos de los tipos dependientes, de hecho, es en el constructor del tipo Π donde se evidencia, por primera vez, la dependiencia en esta teoría, capturando la noción del producto cartesiano generalizado.

La noción de tipo dependiente es análoga al producto cartesiano dependiente, o generalizado de la teoría de conjuntos. En los conjuntos es posible, para cada conjunto A, y una familia de conjuntos $B(x)_{x\in A}$ indexados por A, un nuevo conjunto $\Pi_{x\in A}B_x$, donde sus elementos son funciones con dominion en A, tal que $f(a) \in B_a$ para todo $a \in A$. Consideramos, entonces, $\Pi_{x\in A}B_x$ un producto cartesiano dependiente, pues B_a en el cual $f(a)$ vive, depende de a. De igual forma opera el tipo Π que hemos mencionado en el conjunto de pseudo-términos que conforman un PTS: sea A un tipo definido en Γ y $B(x)$ un tipo, donde x es una variale de tipo A, decimos pues, que $\Pi x : A.B(x)$ es un tipo dependiente, donde para todo x en A, existe una familia de tipos $B(x)$.

Definición 2.2. Especificacion. La especificacion de un PTS es una tripla $S = (S, A, R)$, donde:

1. $\mathcal{S}$ es un conjunto de sorts.
2. $\mathcal{A} \subseteq \mathcal{S} \times \mathcal{S}$ es un conjunto de axiomas.
3. $\mathcal{R} \subseteq \mathcal{S} \times \mathcal{S} \times \mathcal{S}$ es un conjunto de reglas.

Además, se presentan las reglas de formación de un PTS, según lo sugiere Barthe and Coquand:

Nótese que, a diferencia de los sistemas de tipos estudiados hasta ahora, existe una ligera disparidad en el contexto presente en el lambda calculo simplemente tipado y uno dependientemente tipado, y es que, primero, los contextos ahora pueden tener presencia de variables de tipos, de la forma $A : s$ con $s : \mathcal{S}$. Segundo, el orden en el que se insertan juicios $A : s$ en Γ importa para poder manejar de manera correcta y consistente las dependencias, por ejemplo, un juicio de la forma $A : s, \sigma : A$ es totalmente valido, pero $\sigma : A, A : s$ carece de sentido, pues no existe un juicio previo a $\sigma : A$ del que sea posible derivar que A es un tipo bien formado.

Como se ha mencionado ya con anterioridad, la extensión a la que se dedicara el estudio del presente trabajo sera λP, sin embargo, se invita al lector a revisar con más detalles el resto de teorías que, con bastante precisión Barthe and Coquand abordan sobre el Cubo Lambda de Barendregt.

Definición 2.3. Sea $S = \{*, \Box\}$ de sorts y el conjunto $\mathcal{A} = \{(* : \Box)\}$. Es posible obtener λP extendiendo el conjunto de reglas, de forma que $\mathcal{R} = \{(*, *), (*, \Box)\}$. Permitiendo λP introducir los axiomas $\{*, \Box\}$ es posible definir los tipos dependientes, de forma que $B : *$ podria contener una subexpresion $M : A : *$.

La importancia o interes de estudio de los tipos dependientes, radica en como estos habilitan al entendimiento de la correspondencia entre las proposiciones como tipos y los programas como pruebas, a la luz de la logica de predicados. El conjunto de reglas definidas en 2.3 nos muestra que es posible formar expresiones que habitan en $*$ y obtenemos como resultado una expresion que, tambien, habita en $*$. De forma intuitiva, esto corresponde a una funcion ordinaria del tipo $\mathbb{N} \to \mathbb{N} : *$, donde el dominion es de sort *, y el codominio de sortt *. Adicional a ello, el segundo miembro de la tupla, nos indica que podemos formar una expresion que habita en $*$ y como resultado, obtenemos una que habita en $\Box$. Esto ultimo corresponde a la posibilidad de derivaciones como $\Pi x : A.B(x)$ cuando $A : *$ y $B : \Box$, siendo B un tipo indexado por A. Por tanto, la siguiente derivacion es posible:

$$\frac{\Gamma \vdash A : * \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Vec } A \; n : *} \text{ Vec-Form} \tag{2.27}$$

Donde $Vec\, A\, n : *$ habita en $\Box$.

Dado que a los tipos pudiesen contener términos, una noción de igualdad definicional es necesaria en la teoría, por ejemplo, conviene considerar los términos $0 : \mathbb{N}$ y $0 + 0 : \mathbb{N}$ como definicionalmente iguales y, por tanto, no habria distinción entre los términos *Vec(0)* y *Vec 0 + 0*. Si *M : Vec(0)* y *N: Vec 0 + 0*, nos lleva a un juicio de la forma $M = N : \sigma$ que se lee *M y N son terminos definicionalmente iguales en el tipo sigma*, y, además, que $\sigma = \tau : A$, que se lee *sigma y tau son tipos definicionalmente iguales en A*.

Definición 2.4 (Igualdad definicional)**.** Decimos que dos terminos son definicionalmente iguales cuando reducen, luego de cero o mas pasos de reduccion, a un termino en comun Mimram (2020). La igualdad a la que hacemos referencia en este caso, no es igualdad propocional, denotada con $\equiv$, si no, a la igualdad que es inherente del lenguaje base $=$ que corresponde a la equivalencia de terminos bajo β-reduccion.

```
op _+_ : Nat Nat -> Nat .
eq 0 + N = N .
eq s(N) + M = s(N + M) .
```

En este tipo de igualdad, el lenguaje no puede descirnir de dos terminos, pues son exactamente iguales por definicion. Por ejemplo, es posible asertar que *0 + n* y *n* son definicionalmente iguales dada la definicion inductiva de la suma que en Maude proponemos pues *0 + N* reduce a *N* en un solo paso, por tanto, es posible usar *0 + N* y *N* de forma intercambiable.
Sin embargo, esto no siempre es el caso: en contraste *n + 0*, que no es definicionalmente igual a *n* porque *no existe* una ecuacion en nuestra definicion de la suma que inmediatamente pueda asertar que, en este caso, *0 + n* sí reduce a *n*. Pero sí que podemos demostrar proposicionalmente, mediante una prueba más elaborada por induccion en *n*, de que esta igualdad sí se cumple y, finalmente, concluir que *n + 0* $\equiv$ *n*.

Definición 2.5 (Igualdad proposicional)**.** Decimos que dos terminos son proposicionalmente iguales cuando existe un termino p que habita $\mathrm{Id}_A(a, b)$, por tanto, existe una prueba tal que $p : Id(a, b)$ es un testigo de esta igualdad en A. A diferencia de la igualdad definicional, que se obtiene en términos de β-reduccion y es una igualdad inherente del lenguaje, la igualdad proposional debe ser demostrada explícitamente mediante la construcción de un testigo. El constructor canónico del tipo identidad es $\mathrm{refl}_a : \mathrm{Id}_A(a, a)$, que atestigua que todo término es trivialmente igual a sí mismo. Para demostrar igualdades no triviales, se requiere construir un término del tipo correspondiente, tipicamente, por inducción.

Ejemplo 2.11. Considerando el mismo ejemplo de la suma sobre naturales, definida por recursion sobre el primer argumento, la proposicion $0 + n \equiv n$ es una igualdad definicional, pues el termino $0 + n$ reduce directamente a n por la primera clausula, de modo que refl_n basta como testigo. En cambio, la proposicion $n + 0 \equiv n$ no es una igualdad definicional, ya que cuando n es una variable, la expresion $n + 0$ no puede reducirse porque Maude no encuentra una ecuacion que inmediatamente pueda cubrir este caso. Es necesario construir el testigo por induccion sobre n:

- El caso base, donde si $n = 0$, entonces $0 + 0$ reduce a 0, y refl_0 es el testigo.
- El caso inductivo, donde si $n \coloneqq \text{suc}(k)$, entonces $\text{suc}(k)+0$ reduce a $\text{suc}(k+0)$ por la segunda clausula. Aplicando cong_{suc} a la hipotesis inductiva $p_k : k+0 \equiv k$, obtenemos el testigo $\text{cong}_{\text{suc}}(p_k) : \text{suc}(k+0) \equiv \text{suc}(k)$.

En Maude, podemos simular este comportameinto de la siguiente manera, asumiendo que el el tipo de proposicion es un sort cualquiera.

```
sort A B .
subsort A B < Nat .
sort Proof .
op cong : (A -> B) Proof -> Proof .
op refl : -> Proof.

op zero' : Nat -> Proof .
eq zero'(0) = refl .
eq zero'(s(N)) = cong(zero'(N))
```

La existencia de esta prueba es lo que nos permite concluir que $n + 0 \equiv n$ proposicionalmente.

Corolario 2.4. *Dos términos que son definicionalmente iguales, también lo son propocionalmente. Lo contrario no tiene por qué cumplirse.*

2.4.1. Tipos Sigma

Los sistemas de tipos puros, en principio, no proveen constructores más allá del tipo Π, por tanto, el proximo tipo que estudiaremos que también formará parte de la teoria y lenguaje que se formalizará en un capitulo posterior, es el tipo sigma. Representado en el lambda calculo simplemente tipado como $A \times B$, es el tipo de una tupla en el que el primer y segundo componente son de tipo A y B respecivamente, sin embargo, en los tipos dependientes, el segundo componente B pudiese tener una ocurrencia x de tipo A, luego, su tipo, tiene la signatura: $\Sigma x : A.B$, o bien $\langle M, N \rangle_{\Sigma x:A}$, donde M habita en A y N en $B(N)$. Para facilitar o, mas bien, asegurar la decibiliad del prototipo, este tipo estará acompanado de su ascripcion.

El conjunto de pseudo-términos T, se extiende, por tanto, con un nuevo miembro representando los tipos sigma:

$$T = ... \mid \Pi V : T.T \mid \langle T, T \rangle \mid fst\, T \mid snd\, T$$

Donde *fst* y *snd* son *proyecciones* que extraen el primer y segundo componente de la tupa $\langle T, T \rangle$. Sus reglas de formacion y eliminacion, se definen como sigue, como sugiere (Hofmann, 2009):

$$\frac{\Gamma \vdash A : * \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Vec } A\ n : *}\ \Sigma\text{-Form} \tag{2.28}$$

$$\frac{\Gamma \vdash A : * \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Vec } A\ n : *}\ \Sigma\text{-Form} \tag{2.29}$$

Los tipos sigma serán utiles para probar propiedades teoricas de la teoría y, con suerte, comprobar que la formalizacion de la teoria de tipos estudiada en este trabajo cumple su funcion.

2.5. Tipos Cociente

Los cociente son un termino ubicuo en matemáticas, sin embargo, desde la teoría de conjuntos estan presentes y se extiende el concepto con naturalidad en la teoría de tipos. En los conjuntos, es posible definir el cociente de un conjunto S en clases de equivalencia, obteniendo el cociente del conjunto, donde cada elemento que lo conforma es considerado igual, o equivalente, dada una relacion de equivalencia $\sim$. Asi mismo, es posible definir el cociente de un tipo, donde dada una relacion de equivalencia, los elementos son iguales dentro del cociente. Los cocientes nos permiten redefinir, por tanto, la igualdad proposicional de un tipo.

Los tipos cociente son el concepto clave del presente trabajo, son sumamente importantes en la formalización de conceptos matemáticos y el razonamiento de programas. Los cocientes, son un concepto de la teoría de tipos extensional. Martin Löf propuso dos variantes de la teoría de tipos que se diferencian por el tratamiento que dan a la igualdad: la teoria de tipos intencional, y la teoria de tipos extensional. La mayoria de lenguajes de programación de hoy en día, parten de una teoría de tipos intencional porque sus propiedades son decidibles, donde en la teoría de tipos extensional no se garantiza del todo. La razón de esto, es que en la teoría de tipos intensional, la igualdad proposicional definida por tipos de identidad no implica igualdad definicional, y la verificación de tipos es decidible. En la teoría de tipos extensional, por otro lado, la igualdad proposicional se identifica con la igualdad definicional, lo que hace que la verificación de tipos sea indecidible (Li, 2014).

Debido a sus buenas propiedades computacionales, la teoría de tipos intensional es más popular, sin embargo, carece de algunos conceptos extensionales importantes, como la extensionalidad funcional y los tipos cociente, que estaremos abordando.

Muchos asistentes de pruebas y lenguajes con tipos dependientes como Coq, Agda o Lean se apoyan en la teoría intensional. Al ser decidible su igualdad definicional, la verificación de tipos se puede automatizar y todo programa bien tipado es terminante, que son justo las garantías que se espera de una herramienta con la que se pretende razonar de forma fiable. Lo que se paga a cambio es la ausencia de los conceptos extensionales que venimos comentando: la extensionalidad funcional, la identificación de pruebas distintas de una misma proposición y los tipos cociente (Li, 2014).

Podría pensarse que basta con postular estos conceptos como axiomas, pero

esa vía tiene un coste, y es que un axioma se limita a afirmar que cierto término existe, sin decir cómo se calcula, y esa falta de garantía es lo que imposibilita al sistema de tipos determinar si una determinada prueba de igualdad existe o no. Como consecuencia de esto, pues, obtenemos términos cerrados que ya no reducen a ningún valor concreto, con lo que perdemos la canonicidad y, con ella, la razón de haber elegido la teoría intensional desde el principio.

La alternativa que ha resultado viable, consiste en dejar intacta la sintaxis y buscar interpretaciones donde los tipos sean algo más que conjuntos. En esa línea aparecieron durante los años noventa varios modelos, entre ellos los modelos de setoides de Hofmann y de Altenkirch, y el modelo de grupoides de Hofmann y Streicher (Hofmann, 1995). Leer los tipos como grupoides abrió una puerta que otros siguieron hacia estructuras de dimensión cada vez mayor, y en esa misma dirección se sitúa la teoría homotópica de tipos propuesta por Voevodsky, donde los tipos se piensan como espacios, los términos como puntos en dichos espacios y los tipos de identidad como caminos entre ellos. Bajo esa lectura la igualdad deja de ser un predicado auxiliar y pasa a ser un tipo con estructura propia en cada nivel. Estos ultimos detalles, los abordaremos de forma superficial en la teoria de tipos de Martin Lof para complementar la teoria de tipos extensional y dar una breve introduccion a la teoria homotopica de tipos.

Definición 2.6. Una relación de equivalencia $\sim$ en un conjunto A es una relación binaria de signatura $R : A \to A \to Prop$ que cumple las siguientes propiedades:

1. Reflexividad: $\forall a \in A,\ a \sim a$.
2. Simetría: $\forall a, b \in A,\ a \sim b \to b \sim a$.
3. Transitividad: $\forall a, b, c \in A,\ a \sim b \to b \sim c \to a \sim c$.

Por tanto, la clase de equivalencia de un elemento $a \in A$ se denota $[a]$, donde $[a] = \{x \in A \mid a \sim x\}$. El conjunto de clases de equivalencia de A viene denotada como A/R, que se lee *el cociente de A por R*. De forma intuitiva, un tipo cociente A/R es un tipo donde su igualdad ha sido redefinida por una relación de equivalencia $\sim$.

En la teoria de tipos intensional, como se mencionó, no existe la noción de tipos cociente dada su naturaleza de redefinir la igualdad definicional, sin embargo, los setoides se han implementado para capturar el mismo comportamiento.

Los setoides, sugieren Li (2014) y Hofmann (1995) definidos como $(A, \sim)$, están compuestos por:

1. Un tipo $A : Type$.
2. Una relacion binaria $\sim$: $A \to A \to Prop$.
3. Una prueba p de que una funcion f respeta la equivalencia (es decir, que es reflexiva, transitiva y simetrica).

Una funcion $f : A \to B$ respeta la equivalencia $\sim$ si $\forall(x, y : A) \to x \sim y \to f(x) =_B f(y)$.

Sin embargo, en la practica el modelo de los setoides no es para nada practico pues, las pruebas de que una funcion f respeta una equivalencia $\sim$ debe pasarse a toda funcion repetidas veces.

Es necesario mencionar la utilidad practica de los cocientes y, dejar claro que no son un concepto puramente matemtico. Tomese como ejemplo el caso de los naturales representados como un par (a, b), donde con su diferencia $a - b$ permite identificar que muchos de ellos pueden representarse comeo un mismo entero, donde, $(3, 1), (4, 2)y(5, 3)$ representaria el numero 2 y ser, de alguna forma, indistiguibles para el programa. La condicion que los agrupa, $a + b = b + c$ es en este caso, la relacion de equivalencia, formar el cociente dada esta relacion es lo que nos permite que colapsen estos elementos en un solo valor indiscernible por el sistema de tipos.

Los multiconjuntos siguen el mismo patrón. Podemos verlos como listas a las que les hemos quitado la información del orden: partimos del tipo de las listas y cocientamos por la relación que identifica dos de ellas cuando una es permutación de la otra. Así $[1, 2, 2]$ y $[2, 1, 2]$ se vuelven el mismo objeto, pero sin llegar al extremo de un conjunto, porque seguimos recordando cuántas veces aparece cada elemento. Si además olvidamos las repeticiones, cocientando un paso más, recuperamos la noción habitual de conjunto finito.

En definitiva, desde el lado de la programación, disponer de cocientes amplía el repertorio de tipos de datos algebraicos y facilita razonar sobre tipos infinitos y sobre la semántica de programas concurrentes, como sugieren Li (2014) y Hofmann (1995).

Segun lo estudiado por Li (2014), las reglas de formacion de los tipos cociente se definen como sigue:

$$\frac{\Gamma \vdash A : *, \quad \Gamma x, y : A \vdash x \sim y : Prop}{\Gamma \vdash A/\sim} \text{ Q-Form} \tag{2.30}$$

Donde $A/\sim$ es el tipo cociente, dado que A es un tipo bien formado en Γ, x y y habitan en A, y existe una prueba, de tipo $Prop$ que atestigua que $x \sim y$.

Adicionalmente, la regla $Q - intro$ permite inferir que $[a]$ es una clase de equivalencia de tipo $A/\sim$, dado que $a : A$ en Γ

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash [a] : A/\sim} \text{ Q-Intro} \tag{2.31}$$

La regla $Q - Ax$, indica que dada una prueba p de equivalencia de $a \sim b$, y dos elementos a y b de tipo A, nos es posible definir que $[a]$ y $[b]$ son iguales definicionalmente dentro del cociente $A/\sim$, pero, fuera de el, son elementos totalmente distinguibles por el sistema de tipos.

$$\frac{\Gamma \vdash a, b : A \; \Gamma \vdash p : a \sim b}{\Gamma \vdash Qax(p) : [a] =_{A/\sim} [b]} \text{ Q-Ax} \tag{2.32}$$

Además de ello, se propone una regla de eliminación que permite aplicar una función f en $A/\sim$, refereida como *lifting*, si y solo si, la misma respeta la relación de equivalencia (Hofmann, 1995).

$$\frac{\Gamma \vdash B \qquad \Gamma \vdash f : A \to B \qquad \Gamma, a, b : A, p : a \sim b \vdash f^{\sim}(a, b, p) : f(a) =_B f(b) \qquad \Gamma \vdash q : A/\sim}{\Gamma \vdash \hat{f}(q) : B} \text{ (Q-elim)} \quad (2.33)$$

La regla de cómputo $Q - comp$ indica que al aplicar f (asumiendo que f es una función que respeta la equivalencia) a un representante $[a]$ del cociente, obtenemos, pues $f(a)$.

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{Qcomp}(a) : \hat{f}([a]) = f(a)} \text{ Q-comp} \quad (2.34)$$

2.6. Teoría de tipos de Martin-Löf

2.7. Reescritura de términos en Maude

La reescritura de términos constituye, a la luz de los capítulos anteriores, una generalización natural de la β-reducción del λ-cálculo. Donde antes orientábamos un redex de la forma $(\lambda x.\, M)\, N$ hacia $M[x := N]$, ahora disponemos de un conjunto arbitrario de ecuaciones $l = r$, leídas como reglas $l \to r$, y las aplicamos a un término siempre que en él aparezca una instancia del lado izquierdo. El proceso de cómputo se reduce, entonces, a buscar redexes y a contraerlos, exactamente igual que antes, pero en un marco mucho más flexible que permite especificar tipos de datos, funciones y, como veremos, sistemas concurrentes enteros.

Maude es un lenguaje declarativo de alto rendimiento, diseñado y mantenido a lo largo de más de dos décadas por un consorcio en el que participan SRI International, la Universitat Politècnica de València, la Universidad Complutense de Madrid, la Universidad de Málaga, la Universidad de Illinois en Urbana-Champaign y la Universidad de Stanford (Durán et al., 2020; Clavel et al., 2007). Su fundamento teórico es la lógica de reescritura, que admite simultáneamente dos lecturas. Por un lado, es un marco semántico en el que pueden especificarse de modo natural una amplia variedad de sistemas concurrentes, desde redes de Petri hasta cálculos de procesos. Por otro, es un marco lógico en el que se pueden codificar otras lógicas tomando sus reglas de inferencia como reglas de reescritura. La máxima que sintetiza esta dualidad, y que articula buena parte del diseño del lenguaje, es la equivalencia

$$\text{Cómputo} \;=\; \text{Deducción}$$

de modo que un programa Maude no es otra cosa que una teoría de reescritura, y la ejecución del programa coincide con la búsqueda de derivaciones en dicha teoría (Clavel et al., 2007). Esta correspondencia es la que hace de Maude un candidato especialmente adecuado para implementar prototipos ejecutables de sistemas de tipos, pues la sintaxis abstracta del lenguaje que se quiera modelar y sus reglas de tipado pueden trasladarse de manera casi directa a Maude.

2.7.1. Sorts y lógica ecuacional ordenada

La primera diferencia notable respecto del λ-cálculo es que Maude obliga a tipar los términos desde la propia signatura. Cada constante y cada operación declara explícitamente los sorts de sus argumentos y de su resultado, donde por *sort* entendemos lo que en la teoría de tipos llamaríamos un tipo básico. La idea de organizar el universo de discurso en sorts disjuntos proviene de la lógica many-sorted, una variante de la lógica de primer orden en la que el dominio se descompone en una colección indexada de subdominios y cada símbolo de función o predicado especifica el sort de cada uno de sus argumentos (Manzano and Aranda, 2022). Esta restricción sintáctica es liviana en apariencia, pero tiene consecuencias profundas: permite distinguir, por ejemplo, los términos aritméticos de los términos booleanos sin necesidad de codificarlos en un dominio común, y descarta de raíz expresiones sin sentido como sumar un valor de verdad a un número natural.

Maude va, sin embargo, un paso más allá de la lógica many-sorted habitual y adopta la lógica ecuacional *order-sorted*. En esta variante, los sorts forman un conjunto parcialmente ordenado $(S, \leq)$ en el que la relación $s \leq s'$ se interpreta como inclusión de subsort. La idea es la siguiente: si declaramos que $\mathsf{NzNat} \leq \mathsf{Nat}$, todo número natural no nulo es también, automáticamente, un número natural, sin necesidad de coerciones explícitas. Las operaciones pueden además estar sobrecargadas a lo largo de la jerarquía de subsorts, de manera que una misma suma puede declararse a la vez sobre Nat y sobre NzNat, y el sistema seleccionará en cada caso el sort más preciso para el resultado. Esta capacidad de refinamiento, conocida como inferencia del menor sort, es lo que permite, por ejemplo, que la expresión $3/(2-7)$ se parsee con sort genérico $[\mathsf{Rat}]$ y se simplifique después a $-3/5$ con sort NzRat, más restrictivo, una vez completada la evaluación (Clavel et al., 2007).

2.7.2. Módulos funcionales

La unidad básica de especificación en Maude es el módulo. Un módulo funcional, declarado entre las palabras clave `fmod` y `endfm`, agrupa una signatura de sorts y operaciones junto con un conjunto de ecuaciones que se interpretan como reglas de reescritura de izquierda a derecha. Es la unidad análoga, en este lenguaje, a lo que en programación funcional sería un módulo con sus tipos algebraicos y sus funciones puras. Considerese el siguiente ejemplo, en el que especificamos los números naturales en notación de Peano:

```
fmod NAT-PEANO is
  sorts NzNat Nat .
  subsorts NzNat < Nat .
  op 0 : -> Nat [ctor] .
  op s : Nat -> NzNat [ctor] .
  op _+_ : Nat Nat -> Nat .
  vars N M : Nat .
  eq 0 + N = N .
  eq s(N) + M = s(N + M) .
endfm
```

El módulo declara dos sorts, NzNat y Nat, con la inclusión NzNat $\leq$ Nat. La constante 0 se introduce con el atributo `ctor`, que la marca como constructor del tipo, y el sucesor s se declara con el mismo atributo. La suma se especifica con notación infija mediante guiones bajos, y sus dos ecuaciones recogen, casi literalmente, la definición estándar por inducción sobre el primer argumento. Una vez introducido el módulo en el intérprete, podemos solicitar la evaluación de una expresión mediante el comando `reduce`:

```
Maude> reduce s(s(0)) + s(s(s(0))) .
result NzNat: s(s(s(s(s(0)))))
```

El intérprete aplica las ecuaciones como reglas de reescritura hasta alcanzar la forma normal e infiere el menor sort posible para el resultado, en este caso NzNat.

Para que un módulo funcional tenga buenas propiedades operacionales, las ecuaciones deben ser confluentes y operacionalmente terminantes, las mismas dos nociones que estudiábamos en el capítulo del λ-cálculo, ahora reformuladas en el contexto de un sistema de reescritura arbitrario. La terminación operacional garantiza que toda secuencia de reescritura concluye en un número finito de pasos, y la confluencia, que el resultado de la evaluación no depende del orden en que se contraen los redexes. Ambas propiedades, indecidibles en general, pueden chequearse automáticamente con las herramientas del *Maude Formal Environment* (Durán et al., 2020), entre las que destacan `MTT` para terminación y `ChC` para confluencia y suficiente completitud. Bajo estas condiciones, todo término se reduce a una forma normal única, lo que dota al lenguaje de una semántica determinista para sus módulos funcionales.

2.7.3. Módulos de sistema y reescritura no determinista

Junto a los módulos funcionales, Maude provee módulos de sistema, declarados entre `mod` y `endm`. Un módulo de sistema extiende a uno funcional con un conjunto adicional de reglas de reescritura $l \to r$ que modelan transiciones de un sistema, semánticamente distintas de las ecuaciones (Clavel et al., 2007). Las ecuaciones describen *cómo se calcula el estado de un término*, esto es, cómo se simplifica a su forma normal. Las reglas, en cambio, describen *cómo evoluciona el sistema*, qué transiciones puede dar de un estado a otro. La diferencia es que las reglas no se asumen confluentes ni terminantes, sino que codifican el comportamiento intrínsecamente no determinista del sistema. Un mismo término puede reescribirse de maneras distintas y conducir a estados muy diferentes, y eso es justamente lo que se quiere capturar al describir, por ejemplo, las transiciones de un protocolo de comunicación, las jugadas de un juego, o los pasos de una máquina abstracta de evaluación para un lenguaje de programación.

La semántica operacional resultante combina ambos niveles. Ante un término, Maude normaliza primero con las ecuaciones y aplica después una regla al resultado. Para explorar el espacio de estados alcanzables desde un término inicial, Maude provee los comandos `rewrite`, que sigue una traza de reescrituras, y `search`, que explora todos los estados alcanzables en anchura buscando uno que satisfaga un patrón dado (Durán et al., 2020). Este último es de especial utilidad cuando se modela un

sistema de tipos como un módulo de sistema: la búsqueda de derivaciones de tipado se reduce a una búsqueda en el grafo de reescrituras, y el comando devuelve, si la hay, la derivación que justifica el juicio.

2.7.4. Formalización de lenguajes y sistemas de tipos

La capacidad de Maude para definir sintaxis con notación definible por el usuario, junto con su soporte para reescritura y para reescritura no determinista, lo convierten en una plataforma especialmente apropiada para formalizar lenguajes de programación y sistemas de tipos. La idea general es trasladar la sintaxis abstracta del lenguaje a una signatura Σ, las reglas de tipado o de evaluación de la semántica operacional a reglas de reescritura R, y las identidades estructurales sobre los términos, como la α-equivalencia o la igualdad definicional, a ecuaciones E. Esta metodología ha sido aplicada con éxito a lenguajes reales, desde fragmentos de C hasta Java o JavaScript, dando lugar a prototipos no solo ejecutables, sino también analizables con las herramientas del entorno formal de Maude (Durán et al., 2020).

Para hacer visible este traslado en el caso más elemental, consideremos un fragmento del lambda cálculo sin tipos. Especificamos los términos con tres constructores que reflejan directamente la definición inductiva del capítulo anterior:

```
fmod LAMBDA-SYNTAX is
  protected STRING .
  sort Var .
  sort Term .
  subsort Var < Term .

  op var : String -> Var [ctor] .
  op lam : Var Term -> Term [ctor] .
  op app : Term Term -> Term [ctor] .
endfm
```

Aquí Var es subsort de Term, lo que captura sintácticamente el hecho de que toda variable es un término. Tanto la aplicación como la lambda abstracción, son constructres que, proporcionando los arugmentos que exige su signatura será posible producir un sort. Aún así, Maude posee capacidades de expresividad más avanzadas, pues es posible sacar provecho de los operadores infix, permitiendonos redefinir, por ejemplo, la aplicacion como sigue:

```
  op _@_: Term Term -> Term [ctor] .
```

El underscore es un placeholder para los argumentos de la expresion. La β-reducción puede expresarse después como una regla de reescritura en un módulo de sistema que extienda al anterior. Para evitar la captura de variables se necesitaría una operación auxiliar de sustitución, definida por ecuaciones, que renombre las variables ligadas con identificadores frescos cuando sea necesario. Esta es la técnica que adoptaremos a lo largo de esta tesis para implementar el lenguaje dependientemente tipado y, posteriormente, su extensión con tipos cociente.

La fuerza de esta aproximación se aprecia mejor con un ejemplo menos trivial. Los términos de un sistema de tipos a la Church incluuyen, junto a las variables y aplicaciones, anotaciones de tipo en cada uno de sus binders, y las reglas de tipado se expresan en forma de juicios: $\Gamma \vdash M : A$. En Maude, basta con introducir un sort para los contextos, otro para los juicios de tipado, y reglas de reescritura que codifiquen cada regla de inferencia.

Tomando como ejemplo el contexto, el mismo puede definirse como un mapeo de variables a tipos, y tipos a tipos de variable $\mathsf{x} \to \tau$, $\mathsf{Nat} \to \mathcal{U}$. El contexto por tanto, almacena información sobre los tipos bien formados en nuestro programa. De no existir un tipo o termino para un tipo en Γ, decimos que es un término o tipo malformado.

$$\Gamma := \quad \emptyset \tag{2.35}$$

$$\mid \quad (x, \tau) :: \Gamma \tag{2.36}$$

La noción de contexto puede traducirse en Maude, como hemos mencionado, creando un sort para este y las ecuaciones que le corresponden. Guiándonos, pues, de la gramática BNF, el contexto puede bien estar vacío, o extendido. Estas operaciones, pueden ser perfectamente codificadas de la siguiente forma:

```
fmod LAMBDA-SYNTAX is
  ...
  sort Ctx .
  op mt : -> Ctx [ctor] .
  op _;_:_ : Ctx Var Term -> Ctx [ctor] .
  var G : Ctx .
endfm
```

Donde la notación *_;_:_*, es un constructor para un sort de tipo *mt; var x : τ*, que representa el contexto vacío, extendido con $x : \tau$.

Tomando como ejemplo la regla de formación del tipo sigma

$$\frac{\Gamma \vdash A : \mathsf{Type} \quad \Gamma, x : A \vdash B : \mathsf{Type}}{\Gamma \vdash \Sigma\, x : A.\, B : \mathsf{Type}}\ \Sigma\text{-Form} \tag{2.37}$$

Que puede leerse como: *si en el contexto* Γ *el término* A *es un tipo bien formado y, al extender dicho contexto con la hipótesis de que* $x : A$*, es posible inferir que también lo es* B*, entonces* $\Sigma\, x : A.\, B$ *es un tipo en* Γ. El equivalente en Maude de esta regla de formación, es la siguiente:

```
fmod LAMBDA-SYNTAX is
  ...
  op infer : Ctx Term -> [Term] .
  ceq infer(G, Sigma(V, A, B)) = Type
    if Type := norm(infer(G, A))
    /\ Type := norm(infer(G ; V : A, B)) .
endfm
```

Obsérvese que su interpretación no es muy distinta a lo que la regla *(2.27)* formaliza: podemos inferir que $Sigma(V, A, B)$ en la ecuación, es un tipo si se cumple que A es un tipo en Γ y, en el mismo contexto extendido, V es de tipo A, B es también un tipo bien formado.

La ejecución del prototipo, pues, se convierte entonces en la búsqueda de derivaciones, y el comando `search` permite preguntar al sistema si existe una derivación con un consecuente dado, recuperando el término y la sustitución que la justifican (Durán et al., 2020; Clavel et al., 2007).

2.7.5. Reflexión y meta-programación

Un último aspecto que conviene mencionar, dada su utilidad cuando se construye un prototipo de un sistema de tipos, es que la lógica de reescritura es reflexiva (Clavel et al., 2007). Esto quiere decir que existe una teoría universal $\mathcal{U}$, finita y axiomatizable, en la que toda teoría de reescritura $\mathcal{R}$, incluida la propia $\mathcal{U}$, puede representarse como un término $\overline{\mathcal{R}}$, y los pasos de reescritura $t \to t'$ de $\mathcal{R}$ se corresponden con pasos de reescritura $\langle \overline{\mathcal{R}}, \overline{t} \rangle \to \langle \overline{\mathcal{R}}, \overline{t'} \rangle$ en $\mathcal{U}$. Maude implementa esta reflexión en el módulo predefinido `META-LEVEL`, que ofrece operaciones para manipular módulos, términos y reglas como datos de primera clase (Durán et al., 2020). Esto resulta de gran ayuda al implementar verificadores de tipos: el verificador puede inspeccionar el módulo que codifica el lenguaje, aplicar reglas de inferencia a términos meta-representados y devolver derivaciones como resultado, todo dentro de Maude y sin necesidad de salir al lenguaje anfitrión.

En definitiva, Maude provee una forma sumamente natural de implementar reglas de inferencia y la semantica de un lenguaje de manera ecuacional y visual los pasos en los que se computa cada expresión que se suministre en pasos de reducción.

En la próxima sección, una vez establecidas estas herramientas, abordaremos los tipos cociente desde la perspectiva de la teoría de tipos de Martin-Löf, lo que nos permitirá articular en el capítulo siguiente la sintaxis y la semántica operacional del lenguaje dependientemente tipado que constituye el objeto principal de este trabajo, junto con su implementación en Maude apoyada en todas las nociones que hemos introducido aquí.

Capítulo 3

Un Lenguaje Dependientemente Tipado

El lenguaje que se propone en el presente trabajo, parte de un subconjunto del lambda cálculo simplemente tipado, según lo descrito por Löh et al. (2010) en su publicación sobre un lambda cálculo dependientemente tipado, donde nuestra implementación, pese a que difiere ligeramente de la suya, definirá las bases para el prototipo de lenguaje que en las próximas páginas estaremos formalizando. Asimismo, se ha elegido Maude como lenguaje base, por su simpleza para formalizar reglas de inferencia, como se ha podido demostrar. La sintáxis abstracta para términos y tipos, se define como sigue.

3.1. Sintaxis

$$
\begin{aligned}
e, \rho, k := \quad & \alpha && (1)\\
\mid \quad & e :: \rho && (2)\\
\mid \quad & * && (3)\\
\mid \quad & \forall x :: \rho . \rho' && (4)\\
\mid \quad & \lambda x.\, e && (5)\\
\mid \quad & e\ e' && (6)
\end{aligned}
$$

Nótese, como anteriormente se ha cubierto, los términos y los tipos comparten el mismo nivel sintáctico y no existe distinción entre embos, por tanto, pudiese un término ocurrir en un tipo. Con respecto a (2), por motivos prácticos, nuestra implementación no manejará un nodo de anotación explícita, por lo que, en Maude, será totalmente seguro omitirlo.

```
fmod LAMBDA-QTT is
  ...
  sort Var .
  sort Term .
  subsort Var < Term .
  vars N N' : String .
  vars T T1 T1' T2 S A B : Term .
  var V : Var .
  op var : String -> Var [ctor] .
  op Type : -> Term [ctor] .
  op Pi : Var Term Term -> Term [ctor] .
  op lam : Var Term Term -> Term [ctor] .
  op app : Term Term -> Term [ctor] .
  ...
endfm
```

El segmento de constructor de tipos es de mayor interés. Es tentador definir un subsort de Term, tal que, dado un *Type* también podamos obtener un *Term* de forma intercambiable

```
  subsort Type < Term .
```

Pero esto es incorrecto pues, no tiene *Type* un orden estructuralmente menor que *Term* por el contrario, dado que defininimos un cálculo dependiente, términos y tipos vienen siendo lo mismo semánticamente.

3.2. Sustitucion

Las reglas de sustitucion para nuestro lenguaje no son necesariamente distintas de las del lambda cálculo sin tipos discutidas en en un capitulo previo a este, aún así, existen diferencias que son notables en la sustitución del calculo dependiente. Se pretende, por tanto, que las mismas sean triviales para el lector:

1. Las anotaciones de tipo de los binders para el tipo Π, es decir, A en $\lambda x : A.\, t$ y en $\Pi x : A.\, B$, son ahora términos y por tanto pueden contener ocurrencias libres de variables. La sustitución debe aplicarse de forma recursiva sobre el término.

2. Cuando se sustituye bajo un binder de variable y distinta de x, el cuerpo del binder se sustituye normalmente; cuando $y = x$, la sustitución se detiene en la anotación de tipo pero no avanza al cuerpo, pues x queda capturada por el binder.

La regla correspondiente a la sustitucion en el tipo de funcion dependiente es omitida por Löh et al. (2010) sin embargo, es prudente mencionarla en nuestra implementacion.

$$\Pi y : A.B[x := s] = \Pi y : A[x := s].B[x := s]$$

La regla es trivial para λ y Σ que describiremos posteriormente. Para el resto de casos, la sustitución se propaga estrucuturalmente.

Luego, la implementación en Maude define las reglas de cómputo para la sustitución en el cálculo dependientemente tipado, se define una regla por cada constructor del cálculo, definiendo la condición de igualdad o desigualdad sobre los nombres de las variables, expresada de manera ecuacional:

```
fmod LAMBDA-QTT is
  ...
  op subst : Term Var Term -> Term .

  eq subst(var(N), var(N), S) = S .
  ceq subst(var(N), var(N'), S) = var(N) if N =/= N' .
  eq subst(app(T1, T2), V, S) = app(subst(T1, V, S), subst(T2, V, S)) .
  eq subst(Type, V, S) = Type .
  ceq subst(lam(var(N), A, T), var(N), S) = lam(var(N), subst(A, var(N), S)
      , T)
    if true = true .
  ceq subst(lam(var(N), A, T), var(N'), S) =
    lam(var(N), subst(A, var(N'), S), subst(T, var(N'), S))
    if N =/= N' .
  ceq subst(Pi(var(N), A, B), var(N), S) = Pi(var(N), subst(A, var(N), S),
      B)
    if true = true .
  ceq subst(Pi(var(N), A, B), var(N'), S) =
    Pi(var(N), subst(A, var(N'), S), subst(B, var(N'), S))
    if N =/= N' .
  ...
endfm
```

3.3. Normalizacion

Se define, a continuación, la semántica operacional *big step* del proceso de normalización para cada término en el lenguaje, y con a ella, la ecuación que corresponde a cada regla de evaluación.

$$\frac{}{\mathrm{var}(x) \Downarrow \mathrm{var}(x)}\ \mathrm{Var} \qquad \texttt{eq norm(var(N)) = var(N) .}$$

$$\frac{}{\mathsf{Type} \Downarrow \mathsf{Type}}\ \mathrm{Type} \qquad \texttt{eq norm(Type) = Type .}$$

$$\frac{A \Downarrow A' \quad B \Downarrow B'}{\mathrm{Pi(V,\ A,\ B)}}\ \Pi \qquad \texttt{eq norm(Pi(V, A, B)) = Pi(V, norm(A), norm(B)) .}$$

$$\frac{A \Downarrow A' \quad T \Downarrow T'}{\mathrm{lam(V,\ A,\ T)} \Downarrow \mathrm{lam(V,\ A',\ T')}}\ \lambda \qquad \texttt{eq norm(lam(V, A, T)) = lam(V, norm(A), norm(T)).}$$

$$\frac{T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2'}{\text{normApp(V, } T_1, T_2)}\ App \qquad \frac{\text{subst}(T, V, S) \Downarrow T'}{\text{normApp(lam}(V, A, T), S) \Downarrow T'}\ \beta$$

$$\frac{}{\text{normApp}(T_1, T_2) \Downarrow app(\text{T}_1, T_2)}\ App - otherwise \qquad \frac{}{\text{Nat} \Downarrow \text{Nat}}\ Nat$$

$$\frac{}{\text{zero} \Downarrow \text{zero}}\ zero \qquad \frac{T \Downarrow T'}{\text{suc(T)} \Downarrow \text{suc(T')}}\ suc$$

$$\frac{T_1 \Downarrow T'}{\text{natrec}(A, T_1, T_2, \text{zero}) \Downarrow T'}\ \text{natrec-zero}$$

$$\frac{\text{app}(\text{app}(T_2, T), \text{natrec}(A, T_1, T_2, T)) \Downarrow T'}{\text{natrec}(A, T_1, T_2, \text{suc}(T)) \Downarrow T'}\ \text{natrec-suc}$$

$$\frac{A \Downarrow A' \quad T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2' \quad T \Downarrow T'}{\text{natrec}(A, T_1, T_2, T) \Downarrow \text{natrec}(A', T_1', T_2', T')}\ \text{natrec-otherwise}$$

3.4. Verificacion de tipos

Capítulo 4

Extension con tipos cociente

4.1. Formacion

4.2. Introduccion

4.3. Eliminacion

4.4. Computacion

Capítulo 5

Conclusiones

Apéndice A

Código Maude

TODO

Bibliografía

Nipkov Baader. *Term Rewriting and All That*. Cambridge University Press, 2012.

Henk Barendregt, Wil Dekkers, and Richard Statman. *Lambda Calculus with Types*. Perspectives in Logic. Cambridge University Press, 2013.

Henk (Hendrik) Barendregt. An introduction to generalized type systems. *Journal of Functional Programming*, 1:125–154, 04 1991a. doi: 10.1017/S0956796800020025.

Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, revised edition, 1984.

Henk P. Barendregt. *An Introduction To Generalized Type Systems*. Journal of Functional Porgramming. Catholic University of Nijmegen, Netherlands, revised edition, 1991b.

Gilles Barthe and Thierry Coquand. An Introduction to Dependent Type Theory. In Gilles Barthe, Peter Dybjer, Luís Pinto, and João Saraiva, editors, *Applied Semantics*, volume 2395, pages 1–41. Springer Berlin Heidelberg. ISBN 978-3-540-44044-4 978-3-540-45699-5. doi: 10.1007/3-540-45699-6_1. URL `http://link.springer.com/10.1007/3-540-45699-6_1`.

Manuel Clavel, Francisco Durán, Steven Eker, José Meseguer, Patrick Lincoln, Narciso Martí-Oliet, and Carolyn Talcott. *All About Maude – A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science*. Springer, 2007.

Thierry Coquand. Type Theory. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Fall 2022 edition, 2022.

Francisco Durán, Steven Eker, Santiago Escobar, Narciso Martí-Oliet, José Meseguer, Rubén Rubio, and Carolyn Talcott. Programming and symbolic computation in Maude. *Journal of Logical and Algebraic Methods in Programming*, 110: 100497, 2020. doi: 10.1016/j.jlamp.2019.100497.

German Geuvers. *Type Theory and Formal Proof*. Cambride University Press, 2014.

Martin Hofmann. *Extensional Concepts in Intensional Type Theory*. PhD thesis, University of Edinburgh, 1995.

Martin Hofmann. *Syntax and Semantics of Dependent Types*. Cambridge University Press, 2009.

Nuo Li. *Quotient Types in Type Theory*. PhD thesis, University of Nottingham, September 2014.

Andres Löh, Conor McBride, and Wouter Swierstra. A tutorial implementation of a dependently typed lambda calculus. *Fundamenta Informaticae*, 102(2):177–207, 2010.

María Manzano and Victor Aranda. Many-sorted logic. The Stanford Encyclopedia of Philosophy, 2022. URL `https://plato.stanford.edu/entries/logic-many-sorted/`.

Samuel Mimram. *Program = Proof*. Independently published, 2020.

Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.

Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.